

喵喵乐消消-----关卡生成系统重构方案

策略：逆向生成法 (Strategy C)

目标：100% 有解 · 难度分级 · 每局新奇

TL;DR — 三句话读懂这次改动

核心改动

原版：随机填牌 → 重试验证（最多 24 次）→ 不保证有解，通关率依赖运气。

新版：从终态反向构造 → 天然 100% 有解，isSolvable() 降级为开发期告警。

难度通过 buryRate / pathWidth 参数精确控制，无需再改布局逻辑。

1 背景问题

原版 generateBoard() 存在的问题

问题	影响
重试次数硬编码 (12/24)	层数深时概率极低，仍可能生成死局
无解时直接 fallback	Fallback 布局缺乏难度设计，体验割裂
难度无法量化控制	只能靠层配置间接影响，缺精细调节
isSolvable DFS 深度 6	深层复杂布局漏检，假阳性有解判断
每次视觉布局差异小	同一关卡多局新奇感不足

2 逆向生成法原理

逆向生成法的核心洞察：消除序列的逆序，就是一个合法的生成过程。

类比理解：把"解一道题"变成"出一道有答案的题"——先写好答案，再倒推题目。

执行步骤（共 5 步）

1. **Step A** 按关卡配置生成牌型池（类型均匀分配，总数 = 槽位数，保证 3 的倍数）
2. **Step B** 打乱牌型池，每 3 张分为一个「消除组」—— 这组牌将被同时放回棋盘
3. **Step C** 逐组「放回」：为每组找合法槽位，放回后至少 1 张可点击（不被压盖）
4. **Step D** noiseRate 微扰：对低层牌坐标随机微调，保证每局视觉布局不同
5. **Step E** 附加游戏字段（id, type, row, col, layer, zone）输出完整 cards 数组

为什么天然有解？

每组放回时都验证"至少 1 张可点击"，意味着这组牌可以作为当前步骤被消除。

按放回顺序的逆序操作，就是完整的消除路径。

因此生成的布局必然存在一条从头到尾的解路径。

3 新增函数清单

以下 5 个函数全部新增，不修改任何现有函数。

3.1 getDifficultyConfig(levelNum)

根据关卡号返回难度参数，对表中无精确条目的关卡做线性插值。

```
function getDifficultyConfig(levelNum)
  → { buryRate: number, pathWidth: number, noiseRate: number }

// 内置基准点（线性插值，无需手填每一关）：
cycleLevel 1 → buryRate 0.10 pathWidth 3 noiseRate 0.10
cycleLevel 5 → buryRate 0.20 pathWidth 2 noiseRate 0.15
cycleLevel 10 → buryRate 0.25 pathWidth 2 noiseRate 0.18
cycleLevel 15 → buryRate 0.35 pathWidth 2 noiseRate 0.22
cycleLevel 20 → buryRate 0.40 pathWidth 1 noiseRate 0.25
cycleLevel 25 → buryRate 0.50 pathWidth 1 noiseRate 0.30
cycleLevel 30 → buryRate 0.55 pathWidth 1 noiseRate 0.32
```

参数	范围	含义
buryRate	0 ~ 1	深埋率。越高越多牌被放在高层，需要先清上方才能点击，难度上升。
pathWidth	1 ~ 3	路径宽度。每步可选的放置位候选数，1 = 只有一条路（最难），3 = 灵活操作空间大（最易）。
noiseRate	0 ~ 1	噪声率。对低层牌做微小坐标扰动（±0.1），让同一关卡每局视觉布局不同，增加新鲜感。

3.2 buildSlotsForLevel(levelNum, rng)

将原 generateBoard() 内联的槽位构建逻辑抽出为独立函数，供逆向生成复用。

逻辑与原版完全相同，仅抽出到独立函数，返回：

```
→ { slots: Array<{ row, col, layer, zone }>, maxLayer: number }

// 无业务逻辑变化，只是封装，方便 reverseGenerate() 调用。
```

3.3 isCovered(card, cards) / getClickable(cards)

复用原版 isSolvable() 内部的压盖判断语义，抽出为工具函数，供 findPlaceablePositions() 调用。

```
// 压盖条件: layer 更高 且 row/col 差值 < 1 (与原版阈值一致)
isCovered(card, cards) → boolean

// 获取当前场上所有可点击的牌
getClickable(cards) → Array<Card>
```

注意

如果你的原版 isSolvable() 使用的交叠阈值不是 < 1，请将 isCovered() 里的判断改为与原版一致。

该函数只做内部判断，不对外暴露，无需挂到任何模块导出。

3.4 findPlaceablePositions(boardSlots, placed, type, rng, pathWidth, buryThis, maxLayer)

逆向生成的核心子函数：为一组 3 张相同类型的牌，从剩余可用槽位中找出合法的放置位置。

选位策略（优先级从高到低）

6. 放入后至少 1 张处于可点击状态（硬约束，不满足则跳过）
7. buryThis = true → **按层数从高到低排序候选，优先深埋**
8. buryThis = false → **按层数从低到高排序候选，保持路径畅通**
9. 从满足条件的候选池（前 pathWidth × 3 个）中随机选 3 个，增加布局新奇性

```
→ Array<{ row, col, layer, zone }> | null    // 3 个槽位，或 null（极端情况）

// fallback: 若 40 次随机尝试后仍找不到合法位置，
//           放宽约束直接取前 3 个可用槽位（保证生成不卡死）。
```

3.5 reverseGenerate(boardSlots, lv, diffCfg, maxLayer, rng)

逆向生成主体，输入槽位骨架，输出天然有解的完整 cards 数组。

```
Step A  buildTypePool()      - 构造牌型池（每种类型 3k 张，均匀分配）
Step B  shuffleArr(pool)    - 打乱，分为 totalSlots/3 个消除组
Step C  findPlaceablePositions() × N - 逐组放回，深埋/浅放由 buryRate 控制
        深埋概率公式:  $\text{buryRate} \times (1 - \text{gi}/\text{groups.length})$ 
        → 前面的组更容易深埋（后消除），后面的组放浅（先消除，保证路径畅通）
Step D  noiseRate 微扰      - 低层牌坐标  $\pm 0.1$  随机偏移（纯视觉，不影响逻辑）
Step E  附加字段            - id, type, row, col, layer, zone, noiseOffsetRow/Col
```

深埋概率公式说明：第 g_i 组的深埋概率 = $\text{buryRate} \times (1 - g_i/N)$ 。前段组（ g_i 小）深埋概率高，需要玩家先清上方才能点，制造关键路径；后段组（ g_i 大）放浅层，确保游戏末尾路径宽松。

4 修改的函数

4.1 generateBoard(levelNum) ← 主函数

原版改动最大，逻辑流程完全替换。新版伪代码如下：

```
function generateBoard(levelNum) {
  const lv      = getLevelConfig(levelNum)           // 不变
  const rng     = Math.random                       // 不变
  const diffCfg = getDifficultyConfig(levelNum)       // 新增调用

  // ① 槽位骨架（逻辑与原版一致，改为调用抽出的函数）
  const { slots, maxLayer } = buildSlotsForLevel(levelNum, rng)

  // ② 逆向生成 ← 替换原来的重试循环
  let result = reverseGenerate(slots, lv, diffCfg, maxLayer, rng)

  // ③ 添加干扰牌（不变）
  result = addInterferenceCards(result, lv, rng)

  // ④ 构建层样式映射（不变）
  buildLayerOffsetMap(...)

  // ⑤ isSolvable 从"验证器"降级为"开发期告警"
  if (DEV_MODE && !isSolvable(result)) {
    console.warn("addInterferenceCards 破坏了可解性，请检查")
  }

  return result
}
```

删除的代码

原版的 while (attempts < MAX_BOARD_ATTEMPTS) { ... } 重试循环 — 完整删除。

原版的 if (!result) { result = addInterference... } fallback 分支 — 完整删除。

MAX_BOARD_ATTEMPTS 常量 — 不再使用，可删除。

5 不变的函数（零修改）

以下函数无需任何修改，直接保留：

函数名

角色变化

<code>getLevelConfig()</code>	不变
<code>getLayerConfig()</code>	不变
<code>getBoardCoordsByType()</code>	不变
<code>getMaxCardsForBoard()</code>	不变
<code>assignTypesFromPool()</code>	不变 (reverseGenerate 通过 buildTypePool 间接复用其类型分配逻辑)
<code>addInterferenceCards()</code>	不变 (在逆向生成之后调用, 但注意: 干扰牌不应破坏可解性)
<code>buildLayerOffsetMap()</code>	不变
<code>isSolvable()</code>	逻辑不变, 但角色从"主要验证器"降级为"开发期告警"
<code>shuffleArr()</code>	不变

6 接入检查清单

按顺序完成以下操作，改动即生效。

Step 1 确认 lv.types 字段

`reverseGenerate()` 调用 `buildTypePool(lv, totalSlots, rng)`，其中引用 `lv.types`（可用牌型列表，例如 `['cat', 'fish', 'yarn', ...]`）。

如果你的关卡配置对象上没有 `types` 字段，而是在 `assignTypesFromPool()` 里内部构造类型列表，请将 `buildTypePool()` 里第一行改为调用你的函数获取类型列表，其余逻辑不变。

Step 2 确认压盖阈值

`isCovered()` 里使用的交叠判断为 `Math.abs(row 差) < 1 && Math.abs(col 差) < 1`。请确认与你原版 `isSolvable()` 内部的阈值一致，如不一致请同步修改 `isCovered()` 里的判断条件。

Step 3 确认 noiseOffset 渲染

`reverseGenerate()` 在每张牌上附加了 `noiseOffsetRow / noiseOffsetCol` 两个字段（值域 ± 0.1 ）。

渲染层可选择性地将其加到 `row/col` 上，产生视觉微扰效果。如果渲染层不需要，直接忽略这两个字段即可，不影响游戏逻辑。

Step 4 确认 addInterferenceCards 不破坏可解性

`isSolvable()` 现在作为开发期告警，会在 `addInterferenceCards()` 之后运行。若告警触发，说明干扰牌逻辑破坏了有解性，需要检查干扰牌的放置规则，确保它们也满足「放入后至少 1 张可点击」的约束。

Step 5 删除旧代码

- 删除 `while (attempts < MAX_BOARD_ATTEMPTS)` 重试循环及其内部所有代码
- 删除 `if (!result) { fallback... }` 分支
- 可选：删除 `MAX_BOARD_ATTEMPTS` 常量定义

Step 6 调整难度配置（可选）

`DIFFICULTY_CONFIG` 表中的基准点可以按需调整。关卡号之间的值会自动插值，无需手填每一关。如果想对某个特定关卡精调，直接在表中添加条目即可，格式为 `{ buryRate, pathWidth, noiseRate }`。

7 改动前后对比

维度	原版	新版
可解性保证	概率性（重试 12/24 次）	100% 有解（逆向生成）
isSolvable() 角色	主要验证器	开发期告警（辅助）
无解 fallback	直接使用未验证布局	不存在 fallback 分支
难度控制	仅靠层配置间接影响	buryRate / pathWidth 精确控制
每局视觉新奇性	随机但差异小	typePool shuffle + noiseRate 微扰
生成耗时	$O(N^2) \times$ 重试次数	$O(N^2)$ 单次，更快
新增代码量	—	约 200 行（5 个函数）
需要删除代码	—	重试循环 + fallback 约 30 行

8 完整源码附录

将以下代码整体粘贴到你的 `boardGenerator.js`（或对应文件）中，替换 `generateBoard()` 及其调用的内联逻辑。

粘贴位置建议

在文件中找到 `generateBoard()` 函数定义的上方，将 §3 的 5 个新函数一并粘贴进去。

然后用 §4 中的新版 `generateBoard()` 覆盖原函数体。

保持原文件其余部分不变。

```
// —— 新增: 难度配置表 ——
const DIFFICULTY_CONFIG = {
  1: { buryRate: 0.10, pathWidth: 3, noiseRate: 0.10 },
  5: { buryRate: 0.20, pathWidth: 2, noiseRate: 0.15 },
  10: { buryRate: 0.25, pathWidth: 2, noiseRate: 0.18 },
  15: { buryRate: 0.35, pathWidth: 2, noiseRate: 0.22 },
  20: { buryRate: 0.40, pathWidth: 1, noiseRate: 0.25 },
  25: { buryRate: 0.50, pathWidth: 1, noiseRate: 0.30 },

  30: { buryRate: 0.55, pathWidth: 1, noiseRate: 0.32 },
};

function getDifficultyConfig(levelNum) {
  const cycleLevel = (levelNum - 1) % 30 + 1;
  const keys = Object.keys(DIFFICULTY_CONFIG).map(Number).sort((a,b) => a-b);
  if (DIFFICULTY_CONFIG[cycleLevel]) return { ...DIFFICULTY_CONFIG[cycleLevel] };
  if (cycleLevel <= keys[0]) return { ...DIFFICULTY_CONFIG[keys[0]] };

  if (cycleLevel >= keys[keys.length-1]) return { ...DIFFICULTY_CONFIG[keys[keys.length-1]] };
  let lo = keys[0], hi = keys[1];
  for (let i = 0; i < keys.length-1; i++) {
    if (keys[i] <= cycleLevel && cycleLevel < keys[i+1]) { lo=keys[i]; hi=keys[i+1]; break; }
  }
  const t = (cycleLevel-lo)/(hi-lo);
  const a = DIFFICULTY_CONFIG[lo], b = DIFFICULTY_CONFIG[hi];
  return {
    buryRate: a.buryRate + t*(b.buryRate - a.buryRate),
    pathWidth: Math.round(a.pathWidth + t*(b.pathWidth - a.pathWidth)),
    noiseRate: a.noiseRate + t*(b.noiseRate - a.noiseRate),
  };
}

// —— 新增: buildSlotsForLevel ——
function buildSlotsForLevel(levelNum, rng = Math.random) {

  const lv = getLevelConfig(levelNum);
```

```

const { boardType } = lv;
const boardCoords = getBoardCoordsByType(boardType);
const boardCoordCount = boardCoords.length;
const isShapedBoard = boardType !== BOARD_TYPES.standard;
const baseLayerConfig = getLayerConfig(levelNum);
const totalCards = getMaxCardsForBoard(boardType, boardCoordCount, levelNum);
const layerKeysAll = Object.keys(baseLayerConfig).map(Number).sort((a,b)=>a-b);

const totalBase = layerKeysAll.reduce((s,k) => s+baseLayerConfig[k], 0);
const layerConfig = {};
let allocated = 0;
layerKeysAll.forEach((k,i) => {
  if (i === layerKeysAll.length-1) { layerConfig[k] = totalCards - allocated; }
  else { layerConfig[k] = Math.floor(baseLayerConfig[k]*totalCards/totalBase); }
  allocated+=layerConfig[k]; }
});
const maxLayer = layerKeysAll[layerKeysAll.length-1];

const slots = [];
for (let layer = 0; layer <= maxLayer; layer++) {
  const count = layerConfig[layer] || 0;
  const coordPool = isShapedBoard ? shuffleArr([...boardCoords], rng) : boardCoords;
  for (let i = 0; i < count; i++) {
    const coord = isShapedBoard ? coordPool[i % boardCoordCount] : coordPool[Math.floor(rng()*boardCoordCount)];
    slots.push({ row: coord.row, col: coord.col, layer, zone: "default" });
  }
}
return { slots, maxLayer };
}

// — 新增: isCovered / getClickable —————
function isCovered(card, cards) {
  return cards.some(other => {
    if (other === card) return false;
    if (other.layer <= card.layer) return false;
    return Math.abs(other.row-card.row) < 1 && Math.abs(other.col-card.col) < 1;
  });
}
function getClickable(cards) { return cards.filter(c => !isCovered(c, cards)); }

// — 新增: findPlaceablePositions —————

function getAvailableSlots(boardSlots, placed) {
  const usedKeys = new Set(placed.map(c => `${c.row}_${c.col}_${c.layer}`));
  return boardSlots.filter(s => !usedKeys.has(`${s.row}_${s.col}_${s.layer}`));
}

function findPlaceablePositions(boardSlots, placed, type, rng, pathWidth, buryThis, maxLayer) {
  const available = getAvailableSlots(boardSlots, placed);

```

```

    if (available.length < 3) return null;

    const sorted = [...available].sort((a,b) => buryThis ? b.layer-a.layer :
a.layer-b.layer);
    const MAX_TRIES = 40;
    const candidatePool = sorted.slice(0, Math.min(sorted.length, 30));
    for (let attempt = 0; attempt < MAX_TRIES; attempt++) {
        const poolSize = Math.max(3, pathWidth*3);
        const pool = shuffleArr(candidatePool.slice(0, poolSize), rng);
        if (pool.length < 3) break;
        const chosen = pool.slice(0, 3);

        const tempPlaced = [...placed, ...chosen.map(s => ({ ...s, type }))];
        const chosenInTemp = tempPlaced.slice(-3);
        if (chosenInTemp.some(c => !isCovered(c, tempPlaced))) return chosen;
    }
    return sorted.slice(0, 3); // fallback
}

// — 新增: buildTypePool —————

function buildTypePool(lv, totalSlots, rng) {
    const types = lv.types || [];
    if (types.length === 0) return [];
    const groupsNeeded = totalSlots / 3;
    const baseGroups = Math.floor(groupsNeeded / types.length);
    const remainder = groupsNeeded - baseGroups * types.length;
    const shuffledTypes = shuffleArr([...types], rng);
    const pool = [];

    shuffledTypes.forEach((t, i) => {
        const groupCount = baseGroups + (i < remainder ? 1 : 0);
        for (let g = 0; g < groupCount; g++) pool.push(t, t, t);
    });
    return pool;
}

// — 新增: reverseGenerate —————

function reverseGenerate(boardSlots, lv, diffCfg, maxLayer, rng) {
    const { buryRate, pathWidth, noiseRate } = diffCfg;
    const typePool = buildTypePool(lv, boardSlots.length, rng);
    const shuffled = shuffleArr([...typePool], rng);
    const groups = [];
    for (let i = 0; i < shuffled.length; i += 3) groups.push(shuffled[i]);
    const placed = [];
    for (let gi = 0; gi < groups.length; gi++) {

        const type = groups[gi];
        const buryThis = rng() < buryRate * (1 - gi/groups.length);
        const positions = findPlaceablePositions(boardSlots, placed, type, rng,
pathWidth, buryThis, maxLayer);
        if (!positions || positions.length < 3) {

```

```

        getAvailableSlots(boardSlots, placed).slice(0,3).forEach(s =>
placed.push({ ...s, type }));
    } else {
        positions.forEach(s => placed.push({ ...s, type }));
    }

```

```

}
if (noiseRate > 0) {
    placed.forEach(card => {
        if (card.layer === 0 && rng() < noiseRate) {
            card._noiseOffsetRow = (rng()-0.5)*0.2;
            card._noiseOffsetCol = (rng()-0.5)*0.2;
        }
    });
}

```

```

}
return placed.map((card, idx) => ({
    id: idx, row: card.row, col: card.col, layer: card.layer,
    zone: card.zone||"default", type: card.type,
    noiseOffsetRow: card._noiseOffsetRow||0,
    noiseOffsetCol: card._noiseOffsetCol||0,
}));
}

```

```

// —— 修改: generateBoard (替换原函数体) ——
function generateBoard(levelNum) {
    const lv = getLevelConfig(levelNum);
    if (!lv) return [];
    const rng = Math.random;
    const diffCfg = getDifficultyConfig(levelNum);
    const { slots, maxLayer } = buildSlotsForLevel(levelNum, rng);

```

```

    let result = reverseGenerate(slots, lv, diffCfg, maxLayer, rng);
    result = addInterferenceCards(result, lv, rng);
    const totalLayers = maxLayer + 1;
    buildLayerOffsetMap(Array.from({ length: totalLayers }, (_,l) => ({
        layer: l, zIndex: (l+1)*10,
        shadowY: l*2, shadowBlur: 4+l*4, shadowAlpha: 0.20+l*0.05,
    })));
    // isSolvable 降级为开发期告警

```

```

    if (process?.env?.NODE_ENV === "development" && !isSolvable(result)) {
        console.warn(`[generateBoard] L${levelNum}: addInterferenceCards 破坏了解性`);
    }
    return result;
}

```